

```
static struct device_attribute dev_attr_cdev_type =
__ATTR(type, 0444, thermal_cooling_device_type_show, NULL);
static DEVICE_ATTR(thermal_cooling_device_max_state, 0444,
thermal_cooling_device_max_state_show, NULL);
static DEVICE_ATTR(thermal_cooling_device_cur_state, 0444,
thermal_cooling_device_cur_state_show,
thermal_cooling_device_cur_state_store);

static ssize_t
thermal_cooling_device_trip_point_show(struct device *dev,
struct device_attribute *attr, char *buf)
{
    struct thermal_cooling_device_instance *instance;

    instance =
    container_of(attr, struct thermal_cooling_device_instance, attr);

    if (instance->trip == THERMAL_TRIPS_NONE)
        return sprintf(buf, "-1\n");
    else
        return sprintf(buf, "%d\n", instance->trip);
}
```

We know that in Linux, a device is accessed as though it is a file in the filesystem (let's call this a device special file). Now you can guess why it is easy to add a new part by incorporating the hardware-dependent code to support the abstract file interface. We know that, given a large number of different hardware devices, the procedure to write a new device driver should not be complex in any respect.

It should be noted that the Linux kernel employs a buffer cache to effectively deal with block devices, and access to them is through a buffer cache subsystem. This further improves the overall system performance (if you wish to know why, then I would say because it reduces the reads and writes to the hardware devices). You may also notice that each hardware device has a request queue and if the buffer cache is not in a position to 'address' a request from in-memory buffers, the buffer cache adds a request to the device's request queue. And it will remain dormant till the request has been 'addressed'.

Let's now go to the technical side of the process. The buffer cache essentially uses *kflushd* (kernel thread) to write buffer pages out to the device (and to remove them from the cache). If, for instance, a request has been placed before a DD (device driver), then it begins the process by manipulating the device's control and status registers (CSRs). There are broadly three modes to move data from the core part of the system to the peripheral device. They are: polling, direct memory access (DMA), and interrupts. I'm sure that these are familiar terms to even intermediate-level users. But, considering the good number of newbies interested in learning about this topic, let me sum up the concepts. In the first case, the device driver regularly checks the

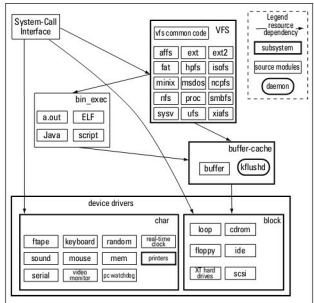


Figure 1: File subsystem structure

CSRs of the peripheral to confirm whether the current request has been addressed. If it is done, it proceeds to the next.

We go for polling when we consider low-speed hardware devices (say a floppy drive). And, finally, in the case of DMA, the DD begins a DMA transfer between the system's main memory and the peripheral. It should be noted that this transfer operation also allows the CPU to work on other tasks along with this. Once the process is over, the CPU receives an interrupt. You may see that this method is far more complicated than those mentioned earlier.

Interrupts

If a device intends to report a change, it does so using an interrupt to the CPU. The same method is used to let the CPU know that a task has been completed. If these are of a high value (logical) then the CPU ceases the current operation and begins to execute the Linux kernel's interrupt handling code. When a particular interrupt is being handled, the other interrupts may be put on hold until the original interrupt is handled. While programming, you need to take note of the efficiency of the handlers, otherwise other interrupts would be lost. Now assume a condition in which the CPU is unable to finish the work in the time frame. Then, it will employ a bottom-half handler to handle the remaining portion of the work. This will be executed by the scheduler in the next phase.

Let's experiment with customising your kernel

You may have plenty of reasons to consider customisation. Advanced users will try to patch the kernel to support hardware that is not currently